Модуль 1. Разведанализ данных, продвинутая визуализация данных

1.1. Знакомство с данными и виды визуализации данных
- Структура работы
- Структура данных
- Импорт библиотек и данных
- Виды визуализации данных

1.2. Разведочный анализ данных (Exploratory Data Analysis, EDA)
- Работа с пропущенными значениями
- Использование описательных статистик
- Промежуточные результаты и выводы
- Функции фильтрации и группировка в рамках разведочного анализа данных
- Объединение таблиц: использование join-функций при визуализации

1.3. Продвинутая визуализация данных, работа с выбросами и аномалиями
- Сравнение долей: использование круговых диаграмм
- Сравнение долей: альтернативный способ
- Работа с качественными характеристиками: создание панели из диаграмм
- Работа с выбросами и аномалиями

Цели модуля

- Познакомимся с основными принципами EDA.
- Повторим основы визуализации.
- Познакомимся с более продвинутыми графиками и их эффективным использованием.
- Разберем основы работы с аномалиями — как их искать и обрабатывать.

1.1. Знакомство с данными и виды визуализации данных

На этой неделе мы будем работать с данными от компании Kiva.

Kiva — это некоммерческая организация, которая позволяет нуждающимся получать финансирование под различные проекты и нужды. Компания существует в более 80 стран мира. Ее девиз — make a loan, change a life.

Источник: [официальный сайт Kiva]

Структура работы

Команда Kiva наняла нас, чтобы проверить наличие аномалий в данных. Также они хотели получить рекомендации о том, что нужно исправить в процессах, чтобы увеличить эффективность привлечения и распределения средств.

Чтобы лучше понимать закономерности в данных, наглядно показать полученные результаты и донести наши мысли заказчику, необходимо сделать большое количество визуализаций. Для этого нужно улучшить навыки владения инструментами визуализации данных.

Нам предстоит вспомнить основы визуализации данных, познакомиться с принципами проведения разведочного анализа данных (Exploratory Data Analysis, EDA) и с основами продвинутой визуализации данных. А также научиться использовать самые распространенные способы поиска аномалий.

Структура данных

kiva_loans — это набор данных, который содержит большую часть информации о займах:

- funded_amount — сумма, выплаченная Kiva агенту на местах (USD);
- loan_amount — сумма, выплаченная полевым агентом заемщику (USD);
- sector — сектор использования займа;
- activity — более гранулированная категория;
- use — назначение использования суммы кредита;
- country — полное название страны, в которой был выдан кредит;
- term_in_months — срок, на который был выдан кредит в месяцах;
- lender_count — общее количество кредиторов, которые внесли свой вклад в этот кредит;
- loaner_genders — разделенные запятыми буквы M, F, где каждый экземпляр представляет одного мужчину или женщину в группе.

kiva_mpi_region_locations — набор данных, содержащий информацию о регионах и показатели MPI (Global Multidimensional Poverty Index, глобальный индекс бедности) ряда регионов (субнациональных) в мире. Чем больше это значение, тем выше бедность в соответствующем регионе.

- country — страна;
- region — регион в стране;
- world_region — часть света;
- MPI — многомерный индекс бедности;
- geo — координаты (широта, долгота).

Импорт библиотек и данных

Мы будем использовать несколько сторонних библиотек:

- DateTime — модуль работы с данными и временем (разберем подробно на второй неделе).
- Matplotlib — базовая библиотека для визуализации на Python.
- NumPy — библиотека работы с векторами и матрицами, которая дает доступ к большему количеству операций и функций.
- Math — библиотека с продвинутыми математическими операциями.
- MissingNo — библиотека детекции и манипуляции пропущенными значениями.
- Seaborn — библиотека продвинутой визуализации. Имеет полную совместимость с Matplotlib, по сути является ее продвинутым братом.
- Os — библиотека работы с операционной системой внутри Python. В нашем случае нужна для манипуляции с папками и файлами.

```
# Импортируем все библиотеки, которые будем использовать в
рамках работы

from datetime import datetime, timedelta

import matplotlib.pyplot as plt

import numpy as np

import pandas as pd

import math

import missingno as msno

import seaborn as sns

import os


%matplotlib inline

palette = plt.get_cmap('Set2')
```

```
# Импортируем данные

kiva_loans =
pd.read_csv('https://raw.githubusercontent.com/ElijahSum/mipt
_visualization/master/week_01_visualization/data/kiva_loans.c
sv', index_col=0)

mpi =
pd.read_csv('https://raw.githubusercontent.com/ElijahSum/mipt
_visualization/master/week_01_visualization/data/kiva_mpi_reg
ion_locations.csv')
```

Посмотрим на внешний вид данных:

```
kiva_loans.head(5)
```

funded_amount	loan_amount	activity	sector	use	country	region	currency	partner_id	term_in_months	lender_count	borrower_genders	
0	300.0	300.0	Fruits & Vegetables	Food	To buy seasonal, fresh fruits to sell.	Pakistan	Lahore	PKR	247.0	12.0	12	female
1	575.0	575.0	Rickshaw	Transportation	to repair and maintain the auto rickshaw used ...	Pakistan	Lahore	PKR	247.0	11.0	14	female, female
2	150.0	150.0	Transportation	Transportation	To repair their old cycle-van and buy another ...	India	Maynaguri	INR	334.0	43.0	6	female
3	200.0	200.0	Embroidery	Arts	to purchase an embroidery machine and a variet...	Pakistan	Lahore	PKR	247.0	11.0	8	female
4	400.0	400.0	Milk Sales	Food	to purchase one buffalo.	Pakistan	Abdul Hakeem	PKR	245.0	14.0	16	female

Из данных видно:

- В нашем распоряжении большое количество информации: размер займа, сфера и цель, на которую выделяется займ, страна, валюта и срок, на который выдается займ.
- Мы можем посмотреть на тип данных каждой переменной. Например, в переменной borrower_genders указаны текстовые данные, но во второй строчке указано female, female. То есть, возможно, переменная небинарная.
- Видим, что заемщики обычно возвращают ту же сумму, что и заняли.

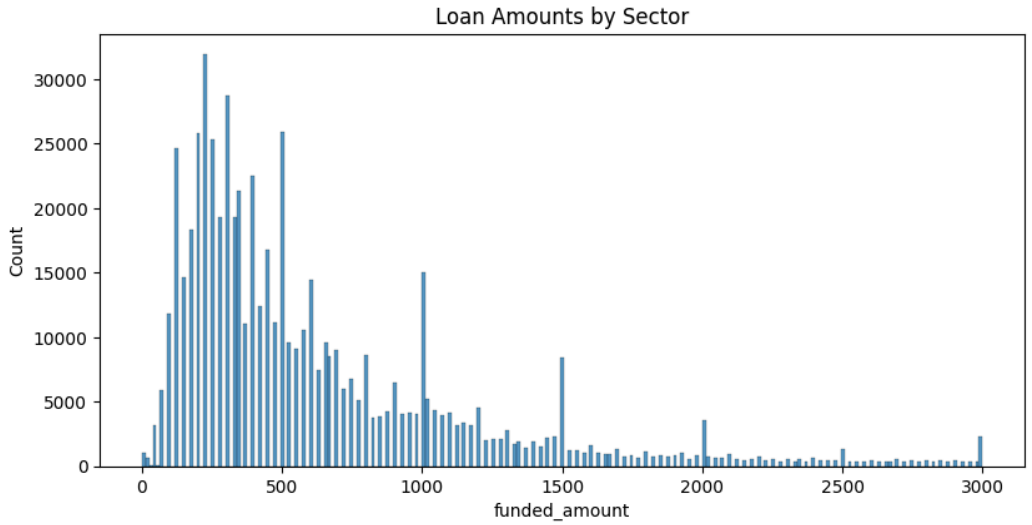
В зависимости от конкретных целей уже из пяти первых строчек датасета можно вынести много информации.

Виды визуализации данных

Реализуем следующие типы графиков:

- гистограмма;
- ящик с усами (Box Plot);
- график количества (Count Plot);
- точечная диаграмма (Scatter Plot).

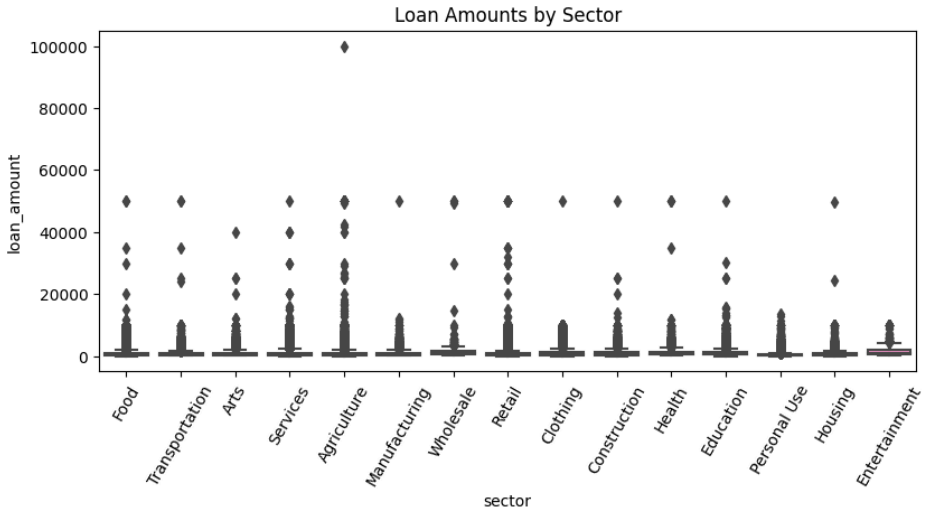
Гистограмма (hist plot):



Гистограмма (histogram plot) — графическое изображение распределения данных по определенным интервалам (бинам) на числовой шкале. Главная цель — показать, как часто значения попадают в каждый из заданных интервалов. Этот тип графика позволяет легко оценить форму и характер распределения данных.

```
# Самые распространенные графики

plt.figure(figsize=(16, 8))



# Hist plot

plt.subplot(2, 2, 1)

sns.histplot(x='funded_amount',
data=kiva_loans[kiva_loans['funded_amount'] <= 3000])

plt.title('Loan Amounts by Sector')



plt.tight_layout()

plt.show()
```

Ящик с усами (box plot):



Ящик с усами (box plot) — графическое изображение распределения данных. График включает прямоугольный ящик, который представляет собой интерквартильный размах (расстояние между первым (25%) и третьим (75%) квартилями), а также «усы», которые выходят из ящика и показывают минимальное и максимальное значения данных, не считая выбросов.

Вот как можно интерпретировать график:

- Ящик:
  - Верхний край ящика — третий квартиль (Q3).
  - Нижний край ящика — первый квартиль (Q1).
  - Высота ящика — интерквартильный размах (IQR).
- Усы:
  - Верхний ус обычно продолжается до максимального значения в пределах 1.5 * IQR от Q3.
  - Нижний ус обычно продолжается до минимального значения в пределах 1.5 * IQR от Q1.
- Выбросы: точки за пределами усов могут считаться выбросами, что указывает на наличие необычных или экстремальных значений.

```
# Самые распространенные графики

plt.figure(figsize=(16, 8))



# Box plot

plt.subplot(2, 2, 2)

sns.boxplot(x='sector', y='loan_amount', data=kiva_loans)

plt.xticks(rotation=60)

plt.title('Loan Amounts by Sector')



plt.tight_layout()

plt.show()
```

График количества (count plot):

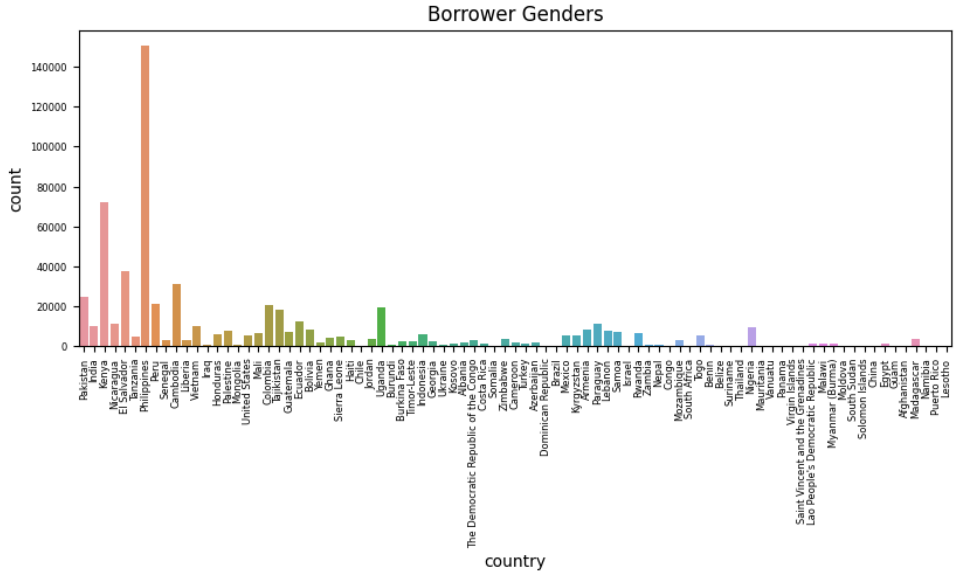
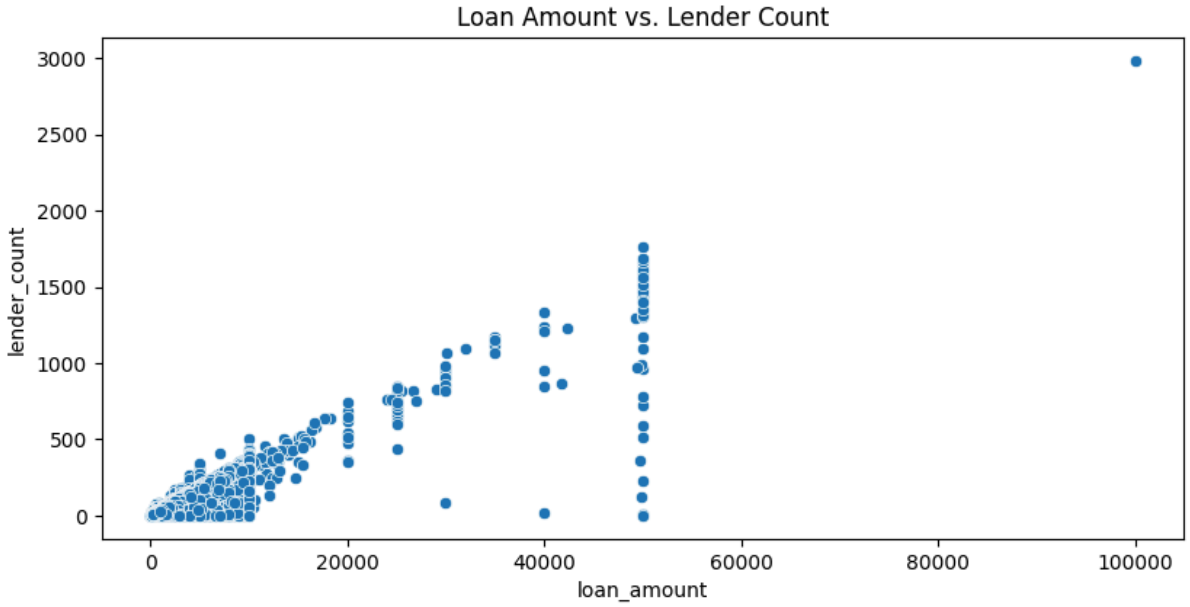


График количества (count plot) — это столбцовый график, отражающий количество наблюдений для каждой уникальной категории. Используется для изучения распределения категориальных переменных.

```
# Самые распространенные графики

plt.figure(figsize=(16, 8))



# Count plot

plt.subplot(2, 2, 3)

sns.countplot(x='country', data=kiva_loans)

plt.xticks(rotation=90)

plt.tick_params(labelsize=6)

plt.title('Borrower Genders')



plt.tight_layout()

plt.show()
```

Точечная диаграмма (scatter plot):



Точечная диаграмма (scatter plot) — это график, отображающий отдельные точки данных в двумерном пространстве. Помогает выявить взаимосвязи и корреляции между двумя переменными.

```
# Самые распространенные графики

plt.figure(figsize=(16, 8))



# Scatter plot

plt.subplot(2, 2, 4)

sns.scatterplot(x='loan_amount', y='lender_count',
data=kiva_loans)

plt.title('Loan Amount vs. Lender Count')



plt.tight_layout()

plt.show()
```

Есть еще несколько распространенных типов графиков, которые не удастся получить, используя эти данные. Самый известный из них — sns.lineplot().

Линейный график (line plot) — визуализация данных в виде линии на координатной плоскости, представляющей тренды и изменения между двумя переменными. Часто используется для отображения временных рядов и выявления вариаций в данных.

Чтобы узнать больше, как реализовывать другие типы графиков, используйте официальную документацию Matplotlib и Seaborn.

1.2. Разведочный анализ данных (Exploratory Data Analysis, EDA)

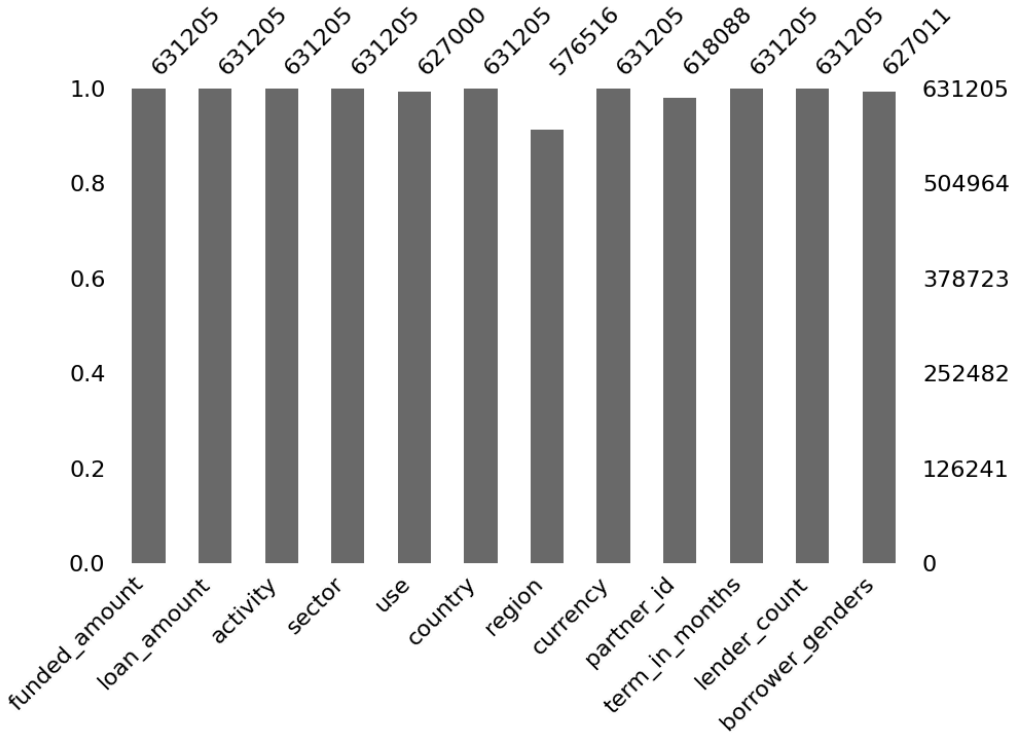
Разведочный анализ данных (Exploratory Data Analysis, EDA) — это этап в анализе данных, направленный на первичное исследование и понимание характеристик набора данных. Он включает использование графиков, статистики и визуализации для выявления паттернов, выбросов и особенностей данных.

EDA выявляет потенциальные направления для более глубокого анализа на начальном этапе исследования данных. Это важный инструмент для выявления закономерностей и формулирования гипотез.

Работа с пропущенными значениями

Посмотрим на наличие пропущенных значений, чтобы не столкнуться с неприятными сюрпризами при дальнейшей работе (например, неправильно обученная модель или невозможность использовать датасет в работе). Это можно сделать с помощью нескольких функций из библиотеки MissingNo:

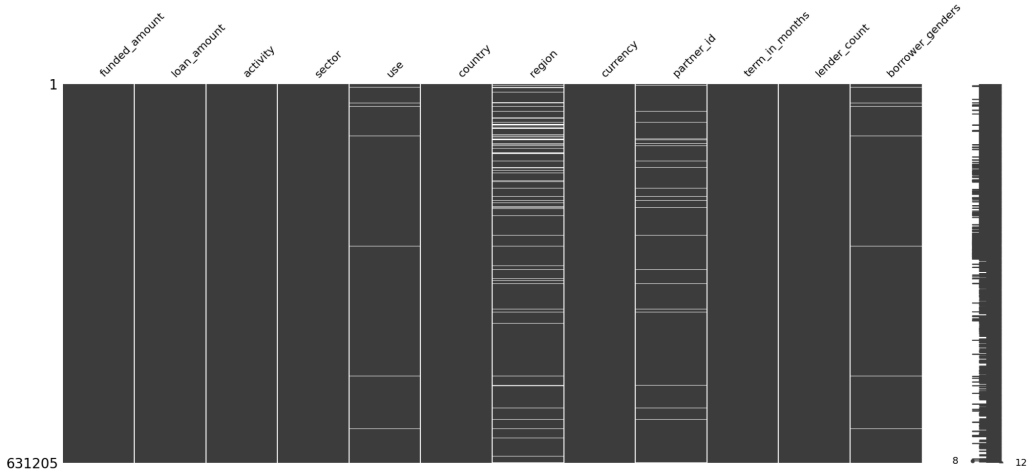
```
msno.bar(kiva_loans, figsize=(10, 6))
```



Мы видим, что больше всего пропусков находится в столбце region. Однако в целом датасет «чистый» — пропуски составляют незначительную долю от общего количества наблюдений (больше 90% наблюдений заполнено).

Место, в котором находится наибольшее количество пропусков, можно распознать с помощью функции msno.matrix():

```
msno.matrix(kiva_loans)
```



Белыми прочерками помечены пропуски в данных. Данные датасета отсортированы по индексу, то есть по порядку.

Пример очень «грязного» датасета с большим количеством пропусков можно найти в статье Bitcoin Historical Data. Чтобы закрепить тему, рекомендуем скачать этот датасет и проанализировать количество пропусков в нем.

Использование описательных статистик

Следующий шаг — изучение атрибутов датасета по отдельности и вывод ряда статистик для каждого из них. Это нужно, чтобы понять, к какому типу относится каждая переменная и как проводился анализ.

Атрибут .dtypes помогает узнать, данные какого типа есть в нашем датасете:

```
kiva_loans.dtypes
```

Результат:

```
funded_amount        float64
loan_amount          float64
activity              object
sector                object
use                   object
country               object
region                object
currency              object
partner_id           float64
term_in_months       float64
lender_count           int64
borrower_genders      object
dtype: object
```

Основные описательные статистики о датасете можно получить с помощью метода .describe():

```
kiva_loans.describe()
```

funded_amount	loan_amount	partner_id	term_in_months	lender_count	
count	631205.000000	631205.000000	618088.000000	631205.000000	631205.000000
mean	797.841533	847.172789	177.147155	13.756108	21.044924
std	1135.536644	1198.432164	93.219670	8.708560	28.642236
min	0.000000	25.000000	9.000000	1.000000	0.000000
25%	250.000000	275.000000	126.000000	8.000000	7.000000
50%	475.000000	500.000000	145.000000	13.000000	13.000000
75%	900.000000	1000.000000	204.000000	14.000000	25.000000
max	100000.000000	100000.000000	526.000000	158.000000	2986.000000

Если нас интересуют только переменные определенного типа (например, object или np.float64), то при использовании метода нужно задать параметр include:

```
kiva_loans.describe(include=[np.float64])
```

funded_amount	loan_amount	partner_id	term_in_months	
count	631205.000000	631205.000000	618088.000000	631205.000000
mean	797.841533	847.172789	177.147155	13.756108
std	1135.536644	1198.432164	93.219670	8.708560
min	0.000000	25.000000	9.000000	1.000000
25%	250.000000	275.000000	126.000000	8.000000
50%	475.000000	500.000000	145.000000	13.000000
75%	900.000000	1000.000000	204.000000	14.000000
max	100000.000000	100000.000000	526.000000	158.000000

```
kiva_loans.describe(include=[object])
```

activity	sector	use	country	region	currency	borrower_genders	
count	631205	631205	627000	631205	576516	631205	627011
unique	163	15	401931	85	12351	67	10982
top	Farming	Agriculture	to buy a water filter to provide safe drinking...	Philippines	Kaduna	PHP	female
freq	67115	168453	4640	150638	9254	150637	400154

Промежуточные результаты и выводы

Разберем первые результаты и выводы, которые мы получили по итогам проведенного анализа.

В этом датасете содержатся данные о 85 странах, где люди запрашивали помощь на финансирование проектов:

```
[ ] kiva_loans.describe(include=[object])
```

	activity	sector	use	country	region	currency	borrower_genders
count	631205	631205	627000	631205	576516	631205	627011
unique	163	15	401931	85	12351	67	10982
top	Farming	Agriculture	to buy a water filter to provide safe drinking...	Philippines	Kaduna	PHP	female
freq	67115	168453	4640	150638	9254	150637	400154

Мы также видим, что Филиппины — страна с самым большим количеством займов:

```
[ ] kiva_loans.describe(include=[object])
```

	activity	sector	use	country	region	currency	borrower_genders
count	631205	631205	627000	631205	576516	631205	627011
unique	163	15	401931	85	12351	67	10982
top	Farming	Agriculture	to buy a water filter to provide safe drinking...	Philippines	Kaduna	PHP	female
freq	67115	168453	4640	150638	9254	150637	400154

Наиболее популярная категория — «Сельское хозяйство»:

```
[ ] kiva_loans.describe(include=[object])
```

	activity	sector	use	country	region	currency	borrower_genders
count	631205	631205	627000	631205	576516	631205	627011
unique	163	15	401931	85	12351	67	10982
top	Farming	Agriculture	to buy a water filter to provide safe drinking...	Philippines	Kaduna	PHP	female
freq	67115	168453	4640	150638	9254	150637	400154

Довольно широкий разброс значений по ряду числовых атрибутов, например, сумма займа, срок погашения кредита:

```
[ ] kiva_loans.describe(include=[np.float64])
```

	funded_amount	loan_amount	partner_id	term_in_months
count	631205.000000	631205.000000	618088.000000	631205.000000
mean	797.841533	847.172789	177.147155	13.756108
std	1135.536644	1198.432164	93.219670	8.708560
min	0.000000	25.000000	9.000000	1.000000
25%	250.000000	275.000000	126.000000	8.000000
50%	475.000000	500.000000	145.000000	13.000000
75%	900.000000	1000.000000	204.000000	14.000000
max	100000.000000	100000.000000	526.000000	158.000000

В этом наборе данных 10982 разных значений для пола:

```
[ ] kiva_loans.describe(include=[object])
```

	activity	sector	use	country	region	currency	borrower_genders
count	631205	631205	627000	631205	576516	631205	627011
unique	163	15	401931	85	12351	67	10982
top	Farming	Agriculture	to buy a water filter to provide safe drinking...	Philippines	Kaduna	PHP	female
freq	67115	168453	4640	150638	9254	150637	400154

Очевидно, что это невозможно. Надо попытаться понять, почему у нас такой набор значений. Помним, что для качественной аналитики важно понимать анализируемый продукт.

Среднее финансирование в мире составляет 797 долларов, а медианное финансирование — 475 долларов:

```
[ ] kiva_loans.describe()
```

	funded_amount	loan_amount	partner_id	term_in_months	lender_count
count	631205.000000	631205.000000	618088.000000	631205.000000	631205.000000
mean	797.841533	847.172789	177.147155	13.756108	21.044924
std	1135.536644	1198.432164	93.219670	8.708560	28.642236
min	0.000000	25.000000	9.000000	1.000000	0.000000
25%	250.000000	275.000000	126.000000	8.000000	7.000000
50%	475.000000	500.000000	145.000000	13.000000	13.000000
75%	900.000000	1000.000000	204.000000	14.000000	25.000000
max	100000.000000	100000.000000	526.000000	158.000000	2986.000000

Возможно, это означает, что есть небольшая доля заявок с крупными суммами. Тогда необходимо поработать с выбросами и попытаться понять, кому и на какие нужды могли понадобиться такие суммы, а также насколько быстро деньги попали к заявителю после публикации заявки.

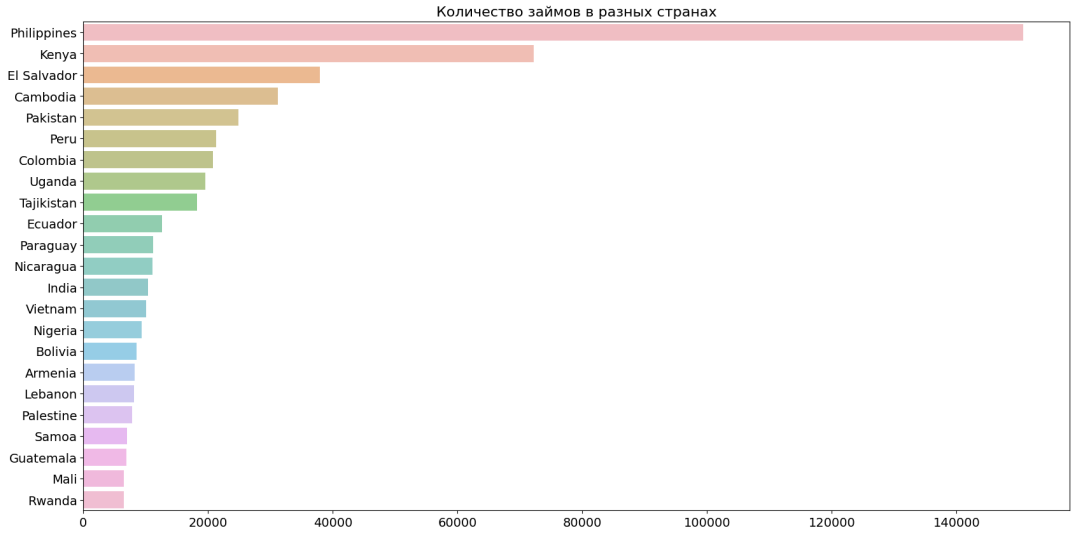
Функции фильтрации и группировка в рамках разведочного анализа данных

Чтобы получить наиболее (статистически) значимые результаты, необходимо думать об используемых нами данных. Важно, чтобы они «не шумели» — не было большого количества ненужных значений, мешающих делать осмысленные выводы.

Во время анализа оставим только записи, которые происходят из стран, составляющих не менее 1% сообщества Kiva. Сгруппируем полученные результаты по странам:

```
countries =
kiva_loans['country'].value_counts()[kiva_loans['country'].va
lue_counts(normalize=True)>=0.01]

list_countries = list(countries.index)
```

```
plt.figure(figsize=(20,10))

plt.title("Количество займов в разных странах", fontsize=16)

plt.tick_params(labelsize=14)

sns.barplot(y=countries.index, x=countries.values, alpha=0.6)

plt.show()
```



Чуть менее четверти всех займов приходится на Филиппины. Возможно, потому, что там широко распространены краткосрочные и небольшие займы. Запомним это предположение и внимательнее рассмотрим его позже. Такая стратегия может оказаться наиболее эффективной для точечного и обширного финансирования в условиях бедности и ограниченных возможностей локальных государств.

Мы также можем заметить, что в списке присутствует множество африканских стран, таких как Кения, Мали, Нигерия, Гана. Но ни одной из стран Европейского Союза или НАФТА.

Сгруппируем страны по частям света. Для этого воспользуемся файлом kiva_mpi_region_locations. Поскольку мы ранее не использовали второй датасет, посмотрим его структуру, чтобы узнать каким образом его группировать.

```
mpi.head()
```

LocationName	ISO	country	region	world_region	MPI	geo	lat	lon	
0	Badakhshan, Afghanistan	AFG	Afghanistan	Badakhshan	South Asia	0.387	(36.7347725, 70.81199529999999)	36.734772	70.811995
1	Badghis, Afghanistan	AFG	Afghanistan	Badghis	South Asia	0.466	(35.1671339, 63.7695384)	35.167134	63.769538
2	Baghlan, Afghanistan	AFG	Afghanistan	Baghlan	South Asia	0.300	(35.8042947, 69.2877535)	35.804295	69.287754
3	Balkh, Afghanistan	AFG	Afghanistan	Balkh	South Asia	0.301	(36.7550603, 66.8975372)	36.755060	66.897537
4	Bamyan, Afghanistan	AFG	Afghanistan	Bamyan	South Asia	0.325	(34.8100067, 67.8212104)	34.810007	67.821210

Поскольку для одной страны указано большое количество регионов, используем join, чтобы вывести среднее по стране, а не по региону страны:

```
mpi_grouped = mpi\
    .groupby(['ISO', 'country', 'world_region'])['MPI']\
    .mean()\
    .fillna(0)\
    .reset_index()
mpi_grouped.head()
```

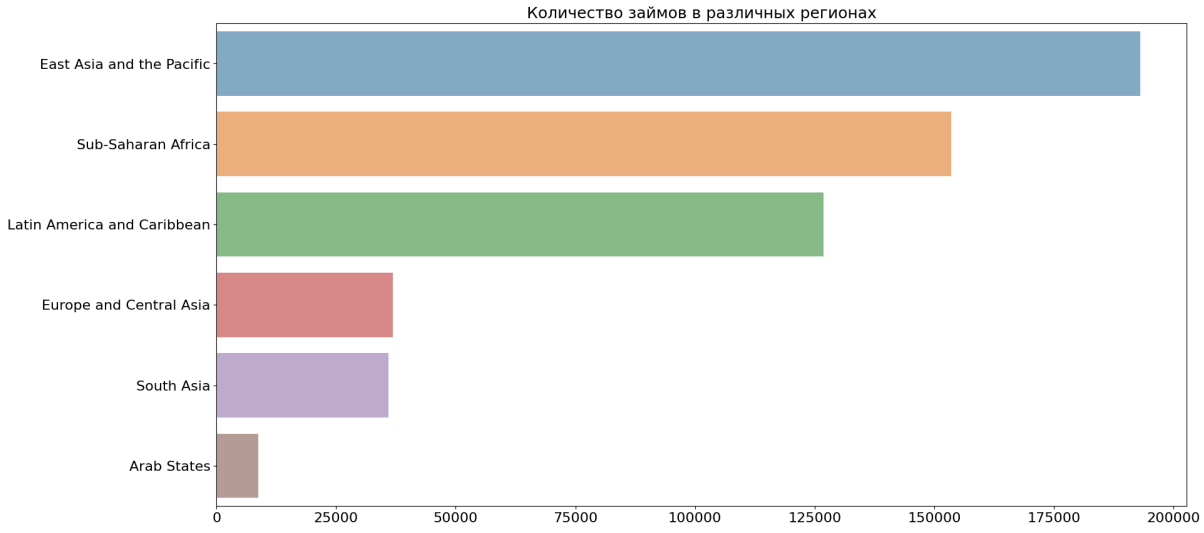
	ISO	country	world_region	MPI
0	AFG	Afghanistan	South Asia	0.309853
1	ALB	Albania	Europe and Central Asia	0.000000
2	ARM	Armenia	Europe and Central Asia	0.000000
3	AZE	Azerbaijan	Europe and Central Asia	0.000000
4	BDI	Burundi	Sub-Saharan Africa	0.411800

Объединение таблиц: использование join-функций при визуализации

```
# Применим left-join для объединения двух таблиц

kiva_loans = kiva_loans.merge(mpi_grouped, how='left',
on='country')
```

```
regions = kiva_loans['world_region'].value_counts()

regions_list = regions.index.to_list()
```

```
plt.figure(figsize=(20,10))

sns.barplot(y=regions.index, x=regions.values, alpha=0.6)

plt.title("Количество займов в различных регионах",
fontsize=18)

plt.tick_params(labelsize=16)

plt.show();
```



После проведения подобного анализа становится очевидным наличие трех лидеров и трех аутсайдеров по количеству выданных займов.

1.3. Продвинутая визуализация данных, работа с выбросами и аномалиями

Теперь перейдем к применению более продвинутых визуальных решений в рамках разведочного анализа данных.

Сравнение долей: использование круговых диаграмм

Посмотрим, люди какого пола обращаются за финансированием:

```
kiva_loans['borrower_genders'].value_counts()
```

Результат:

```
female
400154

male
127314
```

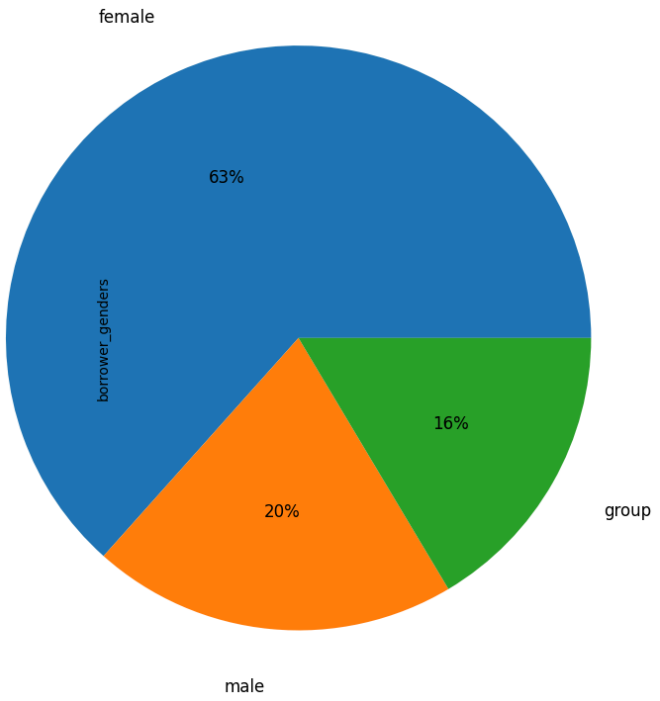
```
female, female
10993

female, female, female
10815

female, female, female, female
8506


...

female, male, female, female, female, male, male, male, male, male, male
1

female, female, female, female, female, male, male, female, male, female,
female, female, female, female
1

female, female, female, female, female, male, female, female, male, female,
female, male, female, female, female, female, female, female, female, female
1

female, male, female, male, male, male, male, male, female, male, male, male,
male, female, female
1

female, male, female, male, male, male, female, male, male, female, female,
female, male, male, male, female, female, male, female, female, female,
female, female, female, male, female, female, female, female, female, male,
male, female, female, female, female, female          1

Name: borrower_genders, Length: 10982, dtype: int64
```

На самом деле, во многих займах заемщик не один человек, а группа людей, у которых есть общий проект. В наборе данных они перечислены как «женщина, женщина, женщина» или «мужчина, женщина». Это объясняет 12 тыс. полов, которые мы обнаружили ранее, — каждая комбинация считается отдельно.

Рассмотрим случай, когда в рамках одного запроса обратилось несколько заемщиков. На круговой диаграмме используем метку group для этих заемщиков. Большинство таких групп смешанные, то есть состоят из представителей обоих размеченных гендеров.

```
kiva_loans['borrower_genders'] = [elem if elem in
['female','male'] else 'group' for elem in
kiva_loans['borrower_genders'] ]
```

```
borrowers = kiva_loans['borrower_genders'].value_counts()

plot = borrowers.plot.pie(fontsize=12, autopct='%1.0f%%',
labeldistance=1.2, radius=2)
```



Выясняется, что большинство заемщиков — женщины, и это весьма неожиданно. Подумайте, что это может означать.

Сравнение долей: альтернативный способ

Чтобы сравнить доли, можно использовать и другой тип графика — horizontal bar plot. Обычно этот график используется при более сложном анализе, когда нужно сравнить доли внутри большого количества групп.

Проведем подобный анализ и ответим на вопрос: «В каких странах подавляющее большинство заемщиков — женщины?»

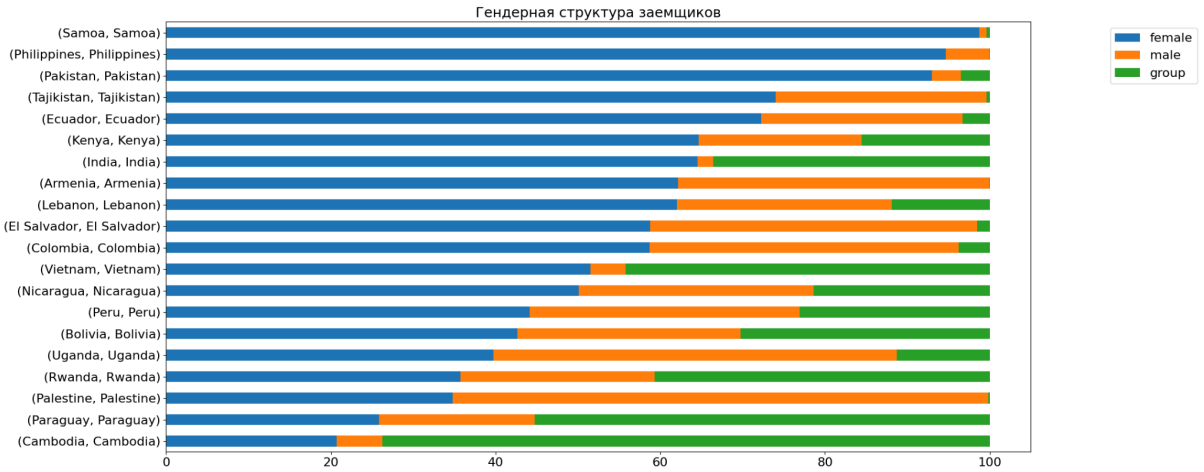
```
# 1. Проверяем, есть ли страна в списке стран
```

```
# 2. Группируем по столбцам country, borrower_genders
# 3. Считаем количество наблюдений в каждой группе
# 4. Находим долю в процентах каждой группы в общей сумме
# 5. Заполним пропуски нулями
# 6. Оставляем только топ-20 значений по женщинам
# 7. Оставляем только интересующие нас значения

gender_by_country =
kiva_loans[kiva_loans['country'].isin(countries.index.values)
]\

   .groupby(['country', 'borrower_genders'],
group_keys=False)['borrower_genders']\

   .count()\

   .groupby(level=0, group_keys=True).apply(lambda x: 100 * x
/ x.sum())\

   .unstack('borrower_genders')\

   .fillna(0)\

   .sort_values(by=['female', 'male'])[-20:]


gender_by_country = gender_by_country[['female', 'male',
'group']]
```

```
# 1. Проверяем, есть ли страна в списке стран
# 2. Группируем по столбцам country, borrower_genders
# 3. Считаем количество наблюдений в каждой группе
# 4. Находим долю в процентах каждой группы в общей сумме
# 5. Убираем группировку по borrower_genders
# 6. Заполним пропуски нулями
# 7. Оставляем только топ-20 значений по женщинам
# 8. Оставляем только интересующие нас значения
```

```
gender_by_country =
kiva_loans[kiva_loans['country'].isin(countries.index.values)
]\

   .groupby(['country', 'borrower_genders'],
group_keys=False)['borrower_genders']\

   .count()\

   .groupby(level=0, group_keys=True).apply(lambda x: 100 * x
/ x.sum())\

   .unstack('borrower_genders')\

   .fillna(0)\

   .sort_values(by=['female', 'male'])[-20:]


gender_by_country = gender_by_country[['female', 'male',
'group']]
```

```
plot = gender_by_country.plot.barh(

       figsize=(20,10)

       , fontsize=16

       , stacked=True

       , title='Гендерная структура заемщиков')

plot.title.set_size(18)

plot.legend(loc=1, bbox_to_anchor=(1.2, 1), fontsize=16)

plot.set(ylabel=None, xlabel=None)

plt.show()
```



С чем может быть связана преобладающая доля женщин в структуре разных стран? Например, с направлениями, которые традиционно ассоциируются с женщинами: точки общественного питания и закусочные, магазины одежды и товаров для дома, бытовые услуги, выращивание сельскохозяйственных культур. Также можем отметить наличие стран, где преобладает ислам — здесь женщине отводится периферийная роль в принятии решений.

Работа с качественными характеристиками: создание панели из диаграмм

Допустим, мы хотим узнать, каковы наиболее популярные виды деятельности, под которые берут кредит у Kiva.

Поскольку в имеющемся датасете две характеристики, отвечающие за род деятельности человека (sector и activity), у нас не получится сделать одну диаграмму. Заметим, что деятельность человека в датасете относится к одному из 15 секторов. Каждый сектор также делится на несколько активностей.

Таким образом, мы можем сделать панель из различных диаграмм. Для этого сгруппируем займы по признакам sector и activity и посмотрим на топ-10 в каждой группе:

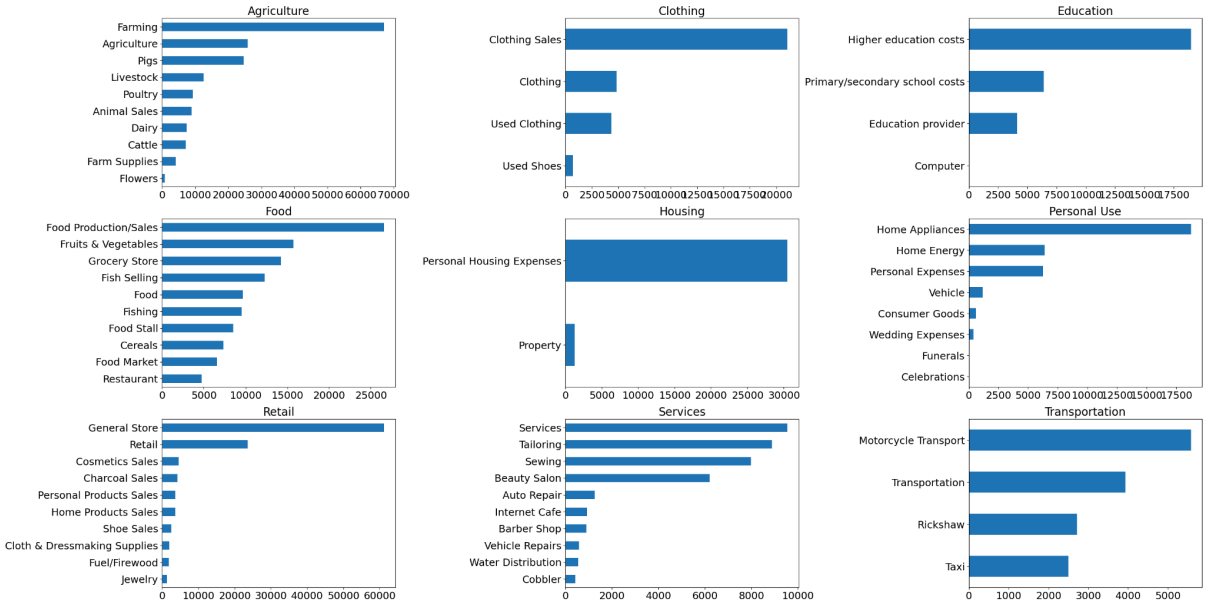
```
sectors = kiva_loans['sector'].value_counts()
```

```
loans_by_activity_sector = \
```

```
kiva_loans[kiva_loans['sector'].isin(sectors[:9].index.values
)]\
   .groupby(['sector', 'activity'])['loan_amount']\
   .count()\
   .reset_index()
```

```
# Создаем поле для графиков размером 3x3, все поле определяем
размером 30 на 15
fig,axes = plt.subplots(3,3, sharex=False, squeeze=False,
figsize=(30,15))


for ax,q in zip(axes.ravel(),
loans_by_activity_sector.sector.unique()):
 # Для каждого из девяти графиков выбираем сектор
 # Внутри каждого сектора оставляем топ-10 популярных
категорий (activities)
   tmp_df =
loans_by_activity_sector[loans_by_activity_sector.sector.eq(q
)]\
               .dropna()\
               .sort_values(by='loan_amount')[-10:]
  # Определяем название каждого графика как название каждого
сектора
   ax.set_title(q, fontsize=20)
   ax.yaxis.label.set_visible(False)
```

```
    plt.tight_layout()

    tmp_df.plot.barh(x='activity', ax=ax, legend=None,
fontsize=18)
```



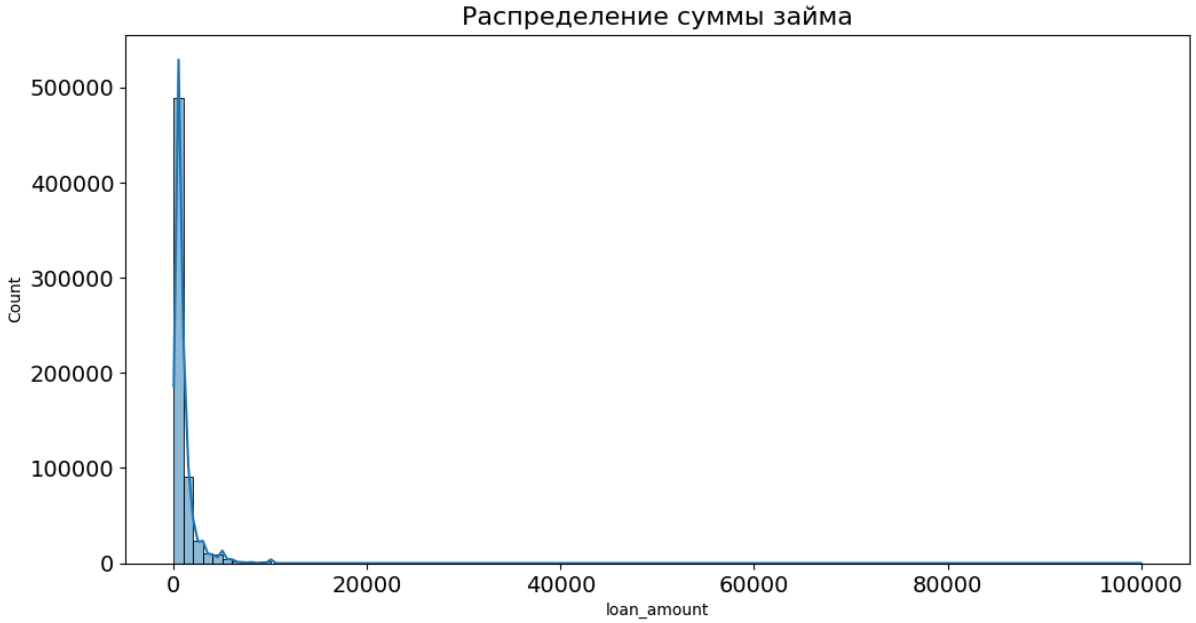
Основной вывод, который мы можем сделать, — львиная доля займов направлена на деятельность с невысокой добавленной стоимостью — сельское хозяйство, розничная торговля, сервис. То есть это деятельность, в которой минимально задействованы смежные секторы экономики и невысок потенциал создания дополнительных рабочих мест и продуктов внутри страны, например, в отличие от производственных предприятий, строительства, ремесленных мастерских, легкой промышленности.

Тем не менее присутствуют и такие направления, как образование: финансирование обучения в школах и университетах, организация и подготовка курсов и образовательных программ.

Работа с выбросами и аномалиями

Обнаружение аномалий в данных. Посмотрим на график распределения сумм займов:

```
plt.figure(figsize=(12,6))

plt.title("Распределение суммы займа", fontsize=16)

plt.tick_params(labelsize=14)



sns.histplot(kiva_loans['loan_amount'], kde=True, bins = 100)

plt.show()
```

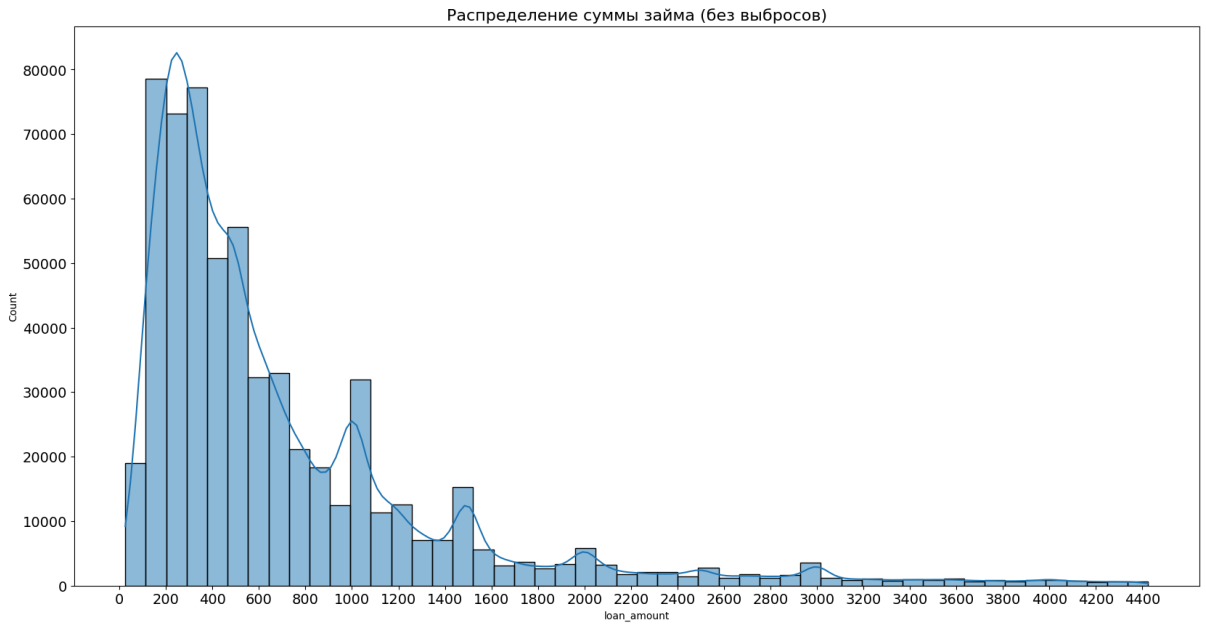


Налицо наличие ряда предельных значений (выбросов). Они искажают распределение, и график не очень информативен в этой форме: нам нужно усечь данные, исключив выбросы из рассмотрения.

Удаление аномалий. Чтобы отсечь выбросы, воспользуемся правилом трех сигм.

Правило трех сигм — это статистический принцип, используемый для оценки разброса данных вокруг среднего значения в распределении. В соответствии с этим правилом, примерно 68% данных находятся в пределах одной стандартной девиации от среднего, 95% — в пределах двух стандартных девиаций, а 99.7% — в пределах трех стандартных девиаций. Таким образом, правило трех сигм позволяет оценить, насколько сосредоточены данные вокруг среднего значения и какой разброс они имеют.

```
# Наблюдаемое отклонение от среднего

dev =
(kiva_loans['loan_amount']-kiva_loans['loan_amount'].mean()).
abs()

# Стандартное отклонение

std = kiva_loans['loan_amount'].std()

# Фильтруем исходный набор данных

kiva_loans_trimmed = kiva_loans[~(dev>3*std)]


plt.figure(figsize=(20,10))

plt.title("Распределение суммы займа (без выбросов)",
fontsize=16)

plt.tick_params(labelsize=14)

plt.xticks(np.arange(0,
max(kiva_loans_trimmed['loan_amount']+1), 200))


sns.histplot(kiva_loans_trimmed['loan_amount'], kde=True,
bins = 50)

plt.show()
```

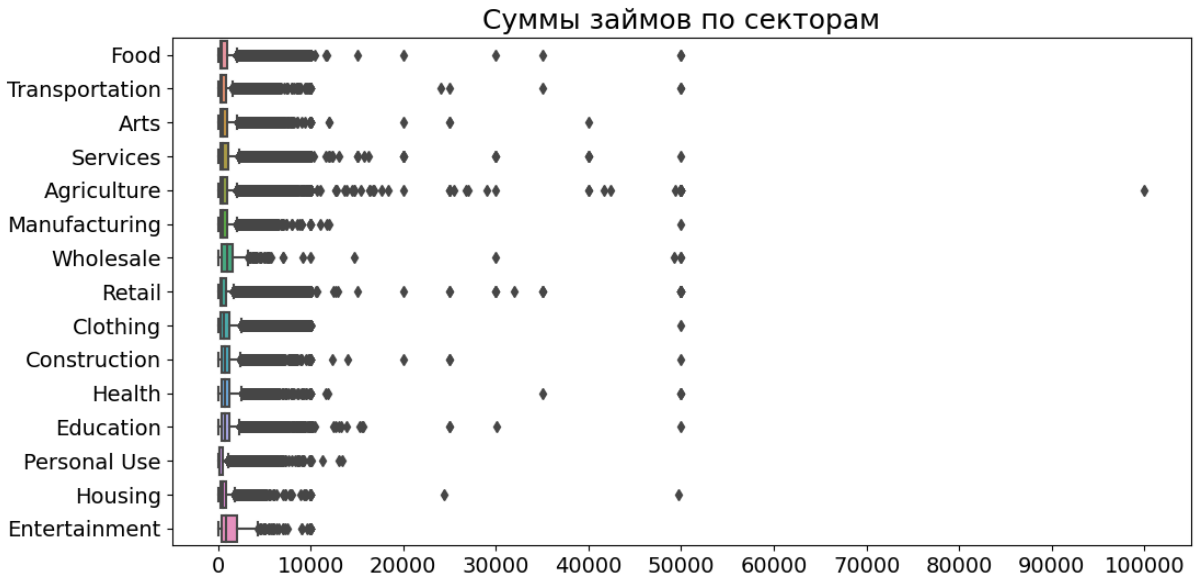


Теперь намного лучше:

- Большинство ссуд составляют от $100 до 600 с пиковыми значениями в районе 200 и 400.
- Сумма естественно уменьшается, но мы можем заметить наличие еще нескольких пиков — $1000 (наиболее четкий), $1500 и $2000.

Примененный нами метод можно использовать и в более комплексных случаях. У нас есть информация о сумме кредита в целом, теперь посмотрим разбиение по секторам:

```
plt.figure(figsize=(12,6))

plt.title("Суммы займов по секторам", fontsize=18)

plt.xticks(np.arange(0, max(kiva_loans['loan_amount']+1),
10000))

plt.tick_params(labelsize=14)



sns.boxplot(x='loan_amount', y="sector",
data=kiva_loans).set(ylabel=None, xlabel=None)

plt.show()
```



Этот график называется «ящик с усами» (box pot). В нашем случае есть совокупность ящиков с усами. Левая граница ящика — первый квантиль (25% значений), правая граница — третий квантиль (75% значений). В свою очередь, «усы» (линии по бокам от ящика) имеют длину в полтора интерквартильных размаха. Все значения за пределом этого интервала считаются выбросами.

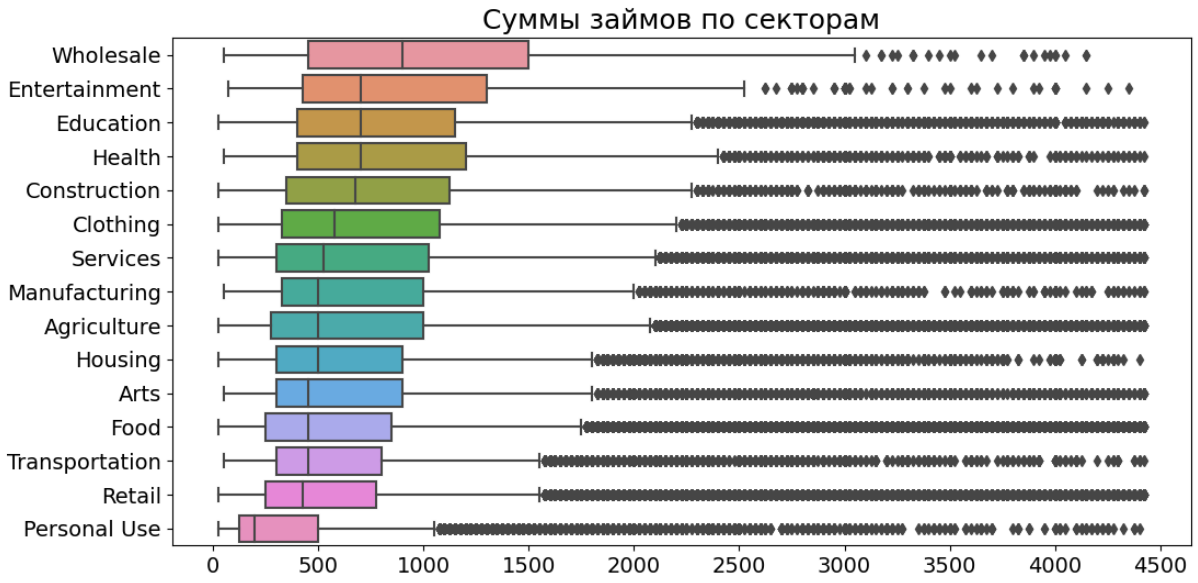
Посмотрим на средние и медианные значения суммы займа по секторам:

```
sector_mean_median = kiva_loans\

    .groupby(['sector'])['loan_amount']\

    .agg(median='median', mean='mean')\

    .sort_values(by='median', ascending=False)



sort_order = sector_mean_median.index.to_list()



sector_mean_median
```

median	mean	
sector		
Wholesale	975.0	1582.059801
Entertainment	900.0	1698.335467
Education	725.0	994.748874
Health	725.0	1071.300975
Construction	700.0	1078.353041
Clothing	600.0	1137.414906
Services	550.0	1073.120142
Manufacturing	525.0	875.216505
Agriculture	500.0	799.362730
Housing	500.0	710.460920
Arts	475.0	1023.745706
Food	450.0	893.096639
Transportation	450.0	719.093285
Retail	425.0	790.511982
Personal Use	200.0	416.904209

После того как мы избавились от выбросов, получаем график:

```
plt.figure(figsize=(12,6))

plt.title("Суммы займов по секторам", fontsize=18)

plt.xticks(np.arange(0, max(kiva_loans['loan_amount']+1),
500))

plt.tick_params(labelsize=14)



sns.boxplot(x='loan_amount', y="sector", order=sort_order,
data=kiva_loans_trimmed).set(ylabel=None, xlabel=None)

plt.show()
```



Прежде всего нужно отметить, что именно медиана дает робастную (устойчивую к выбросам) оценку центрального положения суммы займа. На графике медиана показана как черта внутри ящика.

Из графика ясно, что медиана суммы заявок на личные нужды обладает наименьшим значением. Личные нужды включают предметы обихода и вещи первой необходимости, обеспечение жилища энергией, теплом и водой. Именно эта группа заявок может свидетельствовать о случаях крайней нужды и бедности в различных странах.

На другой стороне графика секторы с самыми большими показателями медианы суммы займа:

- оптовая торговля, требующая значительных затрат на закупку, что вполне логично;
- образование;
- здравоохранение;
- сфера развлечений.

Выводы

Разведочный анализ данных (Exploratory Data Analysis, EDA):

- EDA нужен, чтобы сделать хороший анализ и быстро вникнуть в то, как структурирован датасет.
- Для обнаружения взаимосвязей данных в Pandas используют методы:
    - для группировки — .groupby();
    - для сортировки — .sort_values();
    - для слияния таблиц — .merge().
- Если мы работаем с категориальными данными, для визуализации могут пригодиться гистограммы (plt.hist()), вертикальные и горизонтальные столбчатые (plt.bar() и plt.barh()) и круговые диаграммы (plt.pie()).
- Если мы работаем с численными данными, для визуализации могут пригодиться линейный график (plt.plot()), диаграмма рассеяния (plt.scatter()), ящик с усами (plt.boxplot()) и тепловая карта (seaborn.heatmap()).

Продвинутая визуализация данных:

- Чтобы построить несколько графиков на одном «полотне», нужно использовать параметр subplots = (x, y).
- Для обнаружения пропущенных значений нужно использовать библиотеку MissingNo.

- Для детекции аномалий (выбросов) стоит использовать ящик с усами (нужно обращать внимание на точки за пределами усов).

Работа с выбросами и аномалиями:

- Выбросы могут возникать из-за качественных особенностей данных или ошибок в кодировании.
- Для идентификации выбросов используют правило трех сигм и ящик с усами.
- Ящик с усами можно строить от качественных данных, если в методе sns.box_plot(x = "", y = "") указать их как аргумент y.